

Exercise 2 — Searches & Asymptotic Costs (README)

Array `v[N]` stores `N` integers. We search a value `key` with four functions: 1) sequential linear search on **unordered** array, 2) sequential linear search on **sorted** array, 3) **binary** (iterative) search on sorted array, 4) **binary** (recursive) search on sorted array. Provide the recursive code and compute/justify the asymptotic costs.

Implementations

1) Linear search — unordered

```
int linearSearchUnordered(int v[], int N, int key) {
    for (int i = 0; i < N; ++i) {
        if (v[i] == key) return i;    // found
    }
    return -1;                        // not found
}
```

2) Linear search — sorted (early stop)

```
int linearSearchSorted(int v[], int N, int key) {
    for (int i = 0; i < N; ++i) {
        if (v[i] == key) return i;    // found
        if (v[i] > key) return -1;    // early stop because array is sorted
    }
    return -1;
}
```

3) Binary search — iterative (sorted)

```
int binarySearchIterative(int v[], int N, int key) {
    int lo = 0, hi = N - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;    // avoid overflow
        if (v[mid] == key) return mid;
        if (v[mid] < key) lo = mid + 1; else hi = mid - 1;
    }
}
```

```
    return -1;
}
```

4) Binary search — recursive (sorted) ← requested code

```
int binarySearchRecursive(int v[], int lo, int hi, int key) {
    if (lo > hi) return -1;           // base: not found
    int mid = lo + (hi - lo) / 2;
    if (v[mid] == key) return mid;    // base: found
    if (v[mid] < key) return binarySearchRecursive(v, mid + 1, hi, key);
    else return binarySearchRecursive(v, lo, mid - 1, key);
}
```

Usage: `binarySearchRecursive(v, 0, N-1, key)`

How to Compute the Costs

Below we count comparisons, but any constant work per iteration/level yields the same asymptotics.

A) Linear (unordered)

Work done if the key is at position i is $i+1$ comparisons. Worst case is when it is missing or at the last index: N comparisons. The average over all positions (including “not found”) is proportional to $N/2$.

- **Cost expression:** $T(N) = \Theta(N)$ (since we may examine *every* element).
- **Best / Avg / Worst:** $O(1)$ / $O(N)$ / $O(N)$.

B) Linear (sorted)

We can stop as soon as we see an element $> \text{key}$. In the best case, the first element already exceeds key (1 comparison). If key is larger than all elements or missing beyond the end, we still scan the whole array.

- **Cost expression:** still $T(N) = \Theta(N)$; early-stopping improves constants/average only.
- **Best / Avg / Worst:** $O(1)$ / $O(N)$ / $O(N)$.

C) Binary search (iterative)

Each iteration halves the search interval: $N \rightarrow N/2 \rightarrow N/4 \rightarrow \dots$. After k iterations the size is $N/2^k$. Stop when it drops below 1.

- **Solve for k :** $N/2^k < 1 \Rightarrow k > \log_2 N$, so $k = \lceil \log_2 N \rceil$ iterations.
- **Work per iteration:** constant comparisons and index updates.

Therefore $T(N) = c \cdot \lceil \log_2 N \rceil + O(1) = \Theta(\log N)$.

• **Best / Avg / Worst:** $O(1)$ (middle hit) / $O(\log N)$ / $O(\log N)$.

D) Binary search (recursive)

Let $T(N)$ be the number of operations on an array of size N .

• **Recurrence:** $T(N) = T(N/2) + c$ for $N > 1$, with $T(1) = O(1)$.

• **Solution (repeated substitution / recursion tree):**

$$T(N) = T(N/2) + c$$

$$= T(N/4) + 2c$$

$$= \dots = T(N/2^k) + k \cdot c$$

• Choose k so that $N/2^k = 1 \Rightarrow k = \log_2 N$.

• Hence $T(N) = T(1) + c \cdot \log_2 N = \Theta(\log N)$.

• **Best / Avg / Worst:** $O(1)$ / $O(\log N)$ / $O(\log N)$.

Note: Binary search (both forms) requires the array to be sorted. If sorting is part of the workflow, a comparison sort costs $\Omega(N \log N)$ initially; subsequent searches are $\Theta(\log N)$ each.

Final Asymptotic Complexities

Method	Best	Average	Worst	Final Class
Linear search — unordered	$O(1)$	$O(N)$	$O(N)$	$\Theta(N)$
Linear search — sorted (early stop)	$O(1)$	$O(N)$	$O(N)$	$\Theta(N)$
Binary search — iterative (sorted)	$O(1)$	$O(\log N)$	$O(\log N)$	$\Theta(\log N)$
Binary search — recursive (sorted)	$O(1)$	$O(\log N)$	$O(\log N)$	$\Theta(\log N)$

Intuition

- Linear scans grow **linearly** because, in the worst case, every element must be inspected.
- Binary search grows **logarithmically** because each step discards half of the remaining candidates.

Small Gotchas & Tips

- Guard against integer overflow when computing $\text{mid} = \text{lo} + (\text{hi} - \text{lo})/2$.
- When duplicates exist, the above code returns *one* valid index. To get first/last occurrence, adapt the loop/recursion to continue to the left/right after a hit.

- On already sorted data with *many* searches, binary search is the right tool; on tiny arrays, the constant factors of linear search can be competitive.